

1. Write a program that asks the user how many days are in a month, and what day of the week the month begins on (0 for Monday, 1 for Tuesday, etc), and then prints a calendar for that month.

For example, here is the output for a 31-day month that begins on day 5 (Saturday):

```
days = int(input("Enter the number of days in the month: "))
start_day = int(input("Enter the day of the week the month begins on (0 for Monday, 1 for Tuesday,
etc): "))
30
# Print the header
print(" Mo Tu We Th Fr Sa Su")

# Print leading spaces
for i in range(start_day):
    print(" ", end="")

# Print the days of the month
for day in range(1, days + 1):
    print(f"{day:3}", end="")
    if (start_day + day - 1) % 7 == 0:
        print()

# Print a newline at the end if necessary
if (start_day + days - 1) % 7 != 0:
    print()
```

2. Define a procedure `histogram()` that takes a list of integers and prints a histogram to the screen. For

example, `histogram([4, 9, 7])` should print the following:

```
****
*****
*****
```

```
def histogram(numbers):
    for number in numbers:
        print('*' * number)
```

```
histogram([4, 9, 7])
```

3. Write a version of a palindrome recognizer that also accepts phrase palindromes such as "Go hang a salami I'm a lasagna hog.", "Was it a rat I saw?", "Step on no pets", "Sit on a potato pan, Otis", "Lisa Bonet ate no basil", "Satan, oscillate my metallic sonatas", "I roamed under it as a tired nude Maori", "Rise to vote sir", or the exclamation "Dammit, I'm mad!". Note that punctuation, capitalization, and spacing are usually ignored.

```
def is_palindrome(phrase):
    import re
    cleaned_phrase = re.sub(r'[^a-zA-Z]', '', phrase).lower()
    return cleaned_phrase == cleaned_phrase[::-1]

test_phrases = [
    "Go hang a salami I'm a lasagna hog.",
    "Was it a rat I saw?",
    "Step on no pets",
    "Sit on a potato pan, Otis",
    "Lisa Bonet ate no basil",
    "Satan, oscillate my metallic sonatas",
    "I roamed under it as a tired nude Maori",
    "Rise to vote sir",
    "Dammit, I'm mad!"
]

for phrase in test_phrases:
    if is_palindrome(phrase):
        print(f"'{phrase}' is a palindrome.")
    else:
        print(f"'{phrase}' is not a palindrome.")
```

4. A pangram is a sentence that contains all the letters of the English alphabet at least once, for example: The quick brown fox, jumps over the lazy dog!!!!.

Your task here is to write a function to check a sentence to see if it is a pangram or not.

import string

```
import string

def is_pangram(sentence: str) -> bool:

    # Create a set of all lowercase English alphabet letters
    alphabet = set(string.ascii_lowercase)
    # Convert the sentence to lowercase and check if the alphabet set is a subset
    return alphabet.issubset(sentence.lower())

test_sentence_1 = "The quick brown fox jumps over the lazy dog!!!!."
test_sentence_2 = "Hello, world! This is not a pangram."
```

```
print(is_pangram(test_sentence_1)) # Output: True
print(is_pangram(test_sentence_2)) # Output: False
```

5. In cryptography, a Caesar cipher is a very simple encryption techniques in which each letter in the plain text is replaced by a letter some fixed number of positions down the alphabet. For example, with a shift of 3, A would be replaced by D, B would become E, and so on. The method is named after Julius Caesar, who used it to communicate with his generals. ROT-13 ("rotate by 13 places") is a widely used example of a Caesar cipher where the shift is 13. In Python, the key for ROT-13 may be represented by means of the following dictionary

```
{ 'a': 'n', 'b': 'o', 'c': 'p', 'd': 'q', 'e': 'r', 'f': 's', 'g': 't', 'h': 'u', 'i': 'v', 'j': 'w', 'k': 'x', 'l': 'y', 'm': 'z', 'n': 'a', 'o': 'b', 'p': 'c', 'q': 'd', 'r': 'e', 's': 'f', 't': 'g', 'u': 'h', 'v': 'i', 'w': 'j', 'x': 'k', 'y': 'l', 'z': 'm', 'A': 'N', 'B': 'O', 'C': 'P', 'D': 'Q', 'E': 'R', 'F': 'S', 'G': 'T', 'H': 'U', 'I': 'V', 'J': 'W', 'K': 'X', 'L': 'Y', 'M': 'Z', 'N': 'A', 'O': 'B', 'P': 'C', 'Q': 'D', 'R': 'E', 'S': 'F', 'T': 'G', 'U': 'H', 'V': 'I', 'W': 'J', 'X': 'K', 'Y': 'L', 'Z': 'M' }
```

Your task in this exercise is to implement an encoder/decoder of ROT-13. Once you're done, you will be able to read the following secret message:

Pnrfne pvcure? V zhpu cersre Pnrfne fnynq!

Note that since English has 26 characters, your ROT-13 program will be able to both encode and decode texts written in English.

```
rot13_dict = {
    "a": "n",
    "b": "o",
    "c": "p",
    "d": "q",
    "e": "r",
    "f": "s",
    "g": "t",
    "h": "u",
    "i": "v",
    "j": "w",
    "k": "x",
    "l": "y",
    "m": "z",
    "n": "a",
    "o": "b",
    "p": "c",
    "q": "d",
    "r": "e",
    "s": "f",
    "t": "g",
    "u": "h",
    "v": "i",
    "w": "j",
    "x": "k",
```

```
"y": "l",
"z": "m",
"A": "N",
"B": "O",
"C": "P",
"D": "Q",
"E": "R",
"F": "S",
"G": "T",
"H": "U",
"I": "V",
"J": "W",
"K": "X",
"L": "Y",
"M": "Z",
"N": "A",
"O": "B",
"P": "C",
"Q": "D",
"R": "E",
"S": "F",
"T": "G",
"U": "H",
"V": "I",
"W": "J",
"X": "K",
"Y": "L",
"Z": "M",
}
```

```
def rot13(text):
    result = ""

    for char in text:
        if char in rot13_dict:
            result = result + rot13_dict[char]
        else:
            result = result + char
    return result
```

```
secret_message = "Pnrfne pvcure? V zhpu cersre Pnrfne fnynq!"
decoded_message = rot13(secret_message)
```

```
print("Secret:", secret_message)
print("Decoded:", decoded_message)
```

6. Given a dictionary of students and their favourite colours:

```
people={'Arham':'Blue','Lisa':'Yellow','Vinod':'Purple','Jenny':'Pink'}
```

1. Find out how many students are in the list

2. Change Lisa's favourite colour

3. Remove 'Jenny' and her favourite colour

4. Sort and print students and their favourite colours alphabetically by name

```
# Initial dictionary of students and their favourite colours
```

```
people = {'Arham': 'Blue', 'Lisa': 'Yellow', 'Vinod': 'Purple', 'Jenny': 'Pink'}
```

```
print('--- Initial Dictionary ---')
```

```
print(people)
```

```
print()
```

```
# 1. Find out how many students are in the list
```

```
num_students = len(people)
```

```
print(f'1. Number of students: {num_students}')
```

```
# 2. Change Lisa's favourite colour (e.g., changing it to 'Green')
```

```
people['Lisa'] = 'Green'
```

```
print(f'2. After changing Lisa's colour: {people}')
```

```
# 3. Remove 'Jenny' and her favourite colour
```

```
people.pop('Jenny', None) # or del people['Jenny']
```

```
print(f'3. After removing 'Jenny': {people}')
```

```
# 4. Sort and print students and their favourite colours alphabetically by name
```

```
print('4. Sorted alphabetically by name:')
```

```
for name in sorted(people):
```

```
    print(f' {name}: {people[name]}')
```

7. Write a function `translate()` that will translate a text into "rövarspråket" (Swedish for "robber's language").

That is, double every consonant and place an occurrence of "o" in between.

For example, `translate("this is fun")` should return the string "tothohisos isos fofunon".

```
def translate(text: str) -> str:
```

```
    """Translates a given text into 'rövarspråket' (robber's language)
```

```
    by doubling every consonant and inserting an 'o' in between.
```

```
    """
```

```
    vowels = "aeiouAEIOU"
```

```
    result = []
```

```
    for char in text:
```

```

# Check if the character is an alphabet letter and NOT a vowel
if char.isalpha() and char not in vowels:
    # Double the consonant with an 'o' in between
    result.append(char + "o" + char)
else:
    # Leave vowels, spaces, and punctuation unchanged
    result.append(char)

return "".join(result)

```

```

# --- Example Usage ---
test_text = "this is fun"
translated_text = translate(test_text)

print(f"Original: {test_text}")
print(f"Translated: {translated_text}")

```

8. Write a program that contains a function that has one parameter, n , representing an integer greater than 0. The function should return $n!$ (n factorial). Then write a main function that calls this function with the values 1 through 20, one at a time, printing the returned results. This is what your output should look like:

```

1 1
2 2
3 6
4 24
5 120
6 720
7 5040
8 40320
9 362880
10 628800

```

```

def factorial(n: int) -> int:
    result = 1
    for i in range(2, n + 1):
        result *= i
    return result

def main():
    for i in range(1, 21):
        print(f"{i} {factorial(i)}")

if __name__ == "__main__":
    main()

```

9. Write a recursive sum from 1 to x (i.e. $1 + 2 + \dots + x$) recursively as follows for integer $x \geq 1$:

```
def sum(x):  
    if x == 1:  
        return 1  
    else:  
        return x + sum(x - 1)
```

```
def main():  
    total = sum(10)  
    print(total)
```

```
main()
```

10. Define a function `overlapping()` that takes two lists and returns `True` if they have at least one member in common, `False` otherwise.

```
def sum(x):  
    if x == 1:  
        return 1  
    else:  
        return x + sum(x - 1)
```

```
def main():  
    # Compute and print 1 + 2 + ... + 10  
    total = sum(10)  
    print(total)
```

```
main()
```

11. Write a function `find_longest_word()` that takes a list of words and returns the length of the longest one.

```
def find_longest_word(words: list) -> int:  
    if not words:  
        return 0  
  
    return max(len(word) for word in words)
```

```
word_list = ["apple", "banana", "strawberry", "kiwi"]
longest_length = find_longest_word(word_list)

print(f"The longest word has a length of: {longest_length}")
```

12. Write a function `filter_long_words()` that takes a list of words and an integer `n` and returns the list of words that are longer than `n`

```
def filter_long_words(words: list, n: int) -> list:
    return [word for word in words if len(word) > n]

word_list = ["apple", "banana", "fig", "strawberry", "kiwi", "grape"]
n_value = 5

filtered_words = filter_long_words(word_list, n_value)

print(f"Words longer than {n_value}: {filtered_words}")
```

13. Define a simple "spelling correction" function `correct()` that takes a string and sees to it that

- 1) two or more occurrences of the space character is compressed into one, and
- 2) inserts an extra space after a period if the period is directly followed by a letter.

e.g. `correct("This is very funny and cool.Indeed!")`
should return `"This is very funny and cool. Indeed!"`

```
import re

def correct(text: str) -> str:

    text = re.sub(r"\.([a-zA-Z])", r". \1", text)

    text = re.sub(r"\s+", " ", text)

    return text

sample_text = "This is very funny and cool.Indeed!"
corrected_text = correct(sample_text)

print(f"Original: {sample_text}")
print(f"Corrected: {corrected_text}")
```

14. In English, present participle is formed by adding suffix -ing to infinite form: go -> going. A simple set of heuristic rules can be given as follows:

If the verb ends in e, drop the e and add ing (if not exception: be, see, flee, knee, etc.)

If the verb ends in ie, change ie to y and add ing

For words consisting of consonant-vowel-consonant, double the final letter before adding ing

By default just add ing

Your task in this exercise is to define a function `make_ing_form()` which given a verb in infinitive form

returns its present participle form. Test your function with words such as lie, see, move and hug.

```
def make_ing_form(verb: str) -> str:

    vowels = "aeiou"
    exceptions = ["be", "see", "flee", "knee"]

    if verb in exceptions:
        return verb + "ing"

    elif verb.endswith("ie"):
        return verb[:-2] + "ying"

    elif verb.endswith("e"):
        return verb[:-1] + "ing"

    elif (
        len(verb) >= 3
        and verb[-3] not in vowels
        and verb[-2] in vowels
        and verb[-1] not in vowels
    ):
        return verb + verb[-1] + "ing"

    else:
        return verb + "ing"

test_words = ["lie", "see", "move", "hug", "go"]

print("--- Testing make_ing_form() ---")
for word in test_words:
    print(f"{word} -> {make_ing_form(word)}")
```

15. Write a menu driven program to practice Dictionary functions.

Write a program to accept name of a person and their vehicle and store it in a dictionary.

Ask user if they want to continue to accept multiple values.

Display following menu:

1. Add new person name and a vehicle name.

2. Delete a person name and vehicle name from the dictionary.

----Accept person name from user.

----Check whether person name exists in the dictionary.

----If exists show person name and vehicle name to the user.

----Confirm for deletion, if user enters y
then delete otherwise no. Display appropriate message.

3. Modify vehicle name for the person

----Accept a person name from user.

----Check whether the person's name exists.

----If the name exists, show the person's name and vehicle name to user.

Ask for new value and then overwrite the old value.

4. Search vehicle for the given person's name.

5. Search list of people, who have given a vehicle

6. Display all person names.

7. Display all vehicle names.

```
def main():
```

```
    directory = {}
```

```
    print("--- Initial Setup: Enter Person and Vehicle Data ---")
```

```
    while True:
```

```
        name = input("Enter person's name: ").strip()
```

```
        vehicle = input(f"Enter vehicle for {name}: ").strip()
```

```
        directory[name] = vehicle
```

```
    choice = input("Do you want to add another person? (y/n): ").strip().lower()
```

```
    if choice != "y":
```

```
        break
```

```
# Menu-driven loop
```

```
while True:
```

```
    print("\n===== MENU =====")
```

```
    print("1. Add new person name and a vehicle name")
```

```
    print(
```

```
        "2. Delete a person name and vehicle name (with check and confirmation)"
```

```
    )
```

```
    print("3. Modify vehicle name for a person")
```

```
    print("4. Search vehicle for a given person's name")
```

```
    print("5. Search list of people who have a given vehicle")
```

```
    print("6. Display all person names")
```

```
    print("7. Display all vehicle names")
```

```
    print("8. Exit")
```

```
    option = input("Enter your choice (1-8): ").strip()
```

```

if option == "1":
    name = input("Enter new person's name: ").strip()
    vehicle = input(f"Enter vehicle for {name}: ").strip()
    directory[name] = vehicle
    print(f"Successfully added {name} with vehicle {vehicle}.")

elif option == "2":
    name = input("Enter the person's name to delete: ").strip()
    if name in directory:
        print(f"Found -> Name: {name}, Vehicle: {directory[name]}")
        confirm = (
            input("Are you sure you want to delete this entry? (y/n): ")
            .strip()
            .lower()
        )
        if confirm == "y":
            del directory[name]
            print(f"Record for {name} has been deleted.")
        else:
            print("Deletion cancelled.")
    else:
        print(f"Error: '{name}' does not exist in the directory.")

elif option == "3":
    name = input("Enter the person's name to modify: ").strip()
    if name in directory:
        print(f"Found -> Name: {name}, Current Vehicle: {directory[name]}")
        new_vehicle = input("Enter the new vehicle name: ").strip()
        directory[name] = new_vehicle
        print(f"Vehicle updated successfully for {name}.")
    else:
        print(f"Error: '{name}' does not exist in the directory.")

elif option == "4":
    name = input("Enter person's name to search for their vehicle: ").strip()
    if name in directory:
        print(f"Vehicle for {name}: {directory[name]}")
    else:
        print(f"Error: '{name}' not found in the directory.")

elif option == "5":
    target_vehicle = (
        input("Enter the vehicle name to search people for: ")
        .strip()
        .lower()
    )
    found_people = [
        p
        for p, v in directory.items()
        if v.strip().lower() == target_vehicle
    ]

```

```
if found_people:
    print(f"People who own a '{target_vehicle}': {' '.join(found_people)}")
else:
    print(f"No one found owning the vehicle '{target_vehicle}'.")
```

```
elif option == "6":
    if directory:
        print("All Person Names:")
        for name in directory.keys():
            print(f"- {name}")
    else:
        print("The directory is currently empty.")
```

```
elif option == "7":
    if directory:
        print("All Vehicle Names:")
        for vehicle in directory.values():
            print(f"- {vehicle}")
    else:
        print("The directory is currently empty.")
```

```
elif option == "8":
    print("Exiting program. Goodbye!")
    break
```

```
else:
    print("Invalid choice! Please enter a number between 1 and 8.")
```

```
if __name__ == "__main__":
    main()
```

16. Write a program to display following menu and do the following:
1. Add new city and trees commonly found in the city.
 2. Display all cities and the list of trees for all cities.
 3. Display list of trees of a particular city.
---- Accept a city from user search city and if found display list of trees otherwise
---- Display message not found
 4. Display cities which have the given tree.
---- Accept a tree name from user and display all cities in which the tree is found.
 5. Delete city
---- Accept city from user and delete the city if found.
---- Prompt user before deletion
 6. Modify tree list
---- Accept city and trees to be added in the city. if city exist add trees at the end of the list
---- Otherwise add city and list
 7. Exit

```
def main():
```

```
    city_trees = {}
```

```
    while True:
```

```
        print("\n===== CITY & TREES MENU =====")
```

```
        print("1. Add new city and trees")
```

```
        print("2. Display all cities and their trees")
```

```
        print("3. Display list of trees for a particular city")
```

```
        print("4. Display cities which have a given tree")
```

```
        print("5. Delete a city")
```

```
        print("6. Modify/Add tree list for a city")
```

```
        print("7. Exit")
```

```
    choice = input("Enter your choice (1-7): ").strip()
```

```
    if choice == "1":
```

```
        city = input("Enter city name: ").strip().title()
```

```
        if city in city_trees:
```

```
            print(
                f"'{city}' already exists! Use option 6 to modify its tree list."
            )
```

```
        else:
```

```
            trees_input = input(
```

```
                "Enter trees separated by commas (e.g., Neem, Banyan, Oak): "
```

```
            )
```

```
            # Split by comma and strip whitespace from each tree name
```

```
            trees_list = [t.strip() for t in trees_input.split(",") if t.strip()]
```

```
            city_trees[city] = trees_list
```

```
            print(f"Successfully added '{city}' with trees: {trees_list}")
```

```
    elif choice == "2":
```

```
        if city_trees:
```

```
            print("\n--- Cities and Their Trees ---")
```

```

    for city, trees in city_trees.items():
        print(f"{city}: {' '.join(trees)}")
    else:
        print("No records found in the database.")

elif choice == "3":
    city = input("Enter city name to search: ").strip().title()
    if city in city_trees:
        print(f"Trees found in {city}: {' '.join(city_trees[city])}")
    else:
        print("Not found")

elif choice == "4":
    target_tree = input("Enter tree name to search for: ").strip().lower()
    matching_cities = []
    for city, trees in city_trees.items():
        # Check if the target tree matches any tree in the list (case-insensitive)
        if any(target_tree == t.lower() for t in trees):
            matching_cities.append(city)

    if matching_cities:
        print(
            f"Cities having '{target_tree.title()}':"
            f" {' '.join(matching_cities)}"
        )
    else:
        print(f"No cities found containing the tree '{target_tree.title()}'.")

elif choice == "5":
    city = input("Enter city name to delete: ").strip().title()
    if city in city_trees:
        print(f"Found city '{city}' with trees: {' '.join(city_trees[city])}")
        confirm = (
            input("Are you sure you want to delete this city? (y/n): ")
            .strip()
            .lower()
        )
        if confirm == "y":
            del city_trees[city]
            print(f"City '{city}' has been deleted successfully.")
        else:
            print("Deletion cancelled.")
    else:
        print("City not found.")

elif choice == "6":
    city = input("Enter city name: ").strip().title()
    trees_input = input(
        "Enter trees to be added (separated by commas): "
    )
    new_trees = [t.strip() for t in trees_input.split(",") if t.strip()]

```

```
if city in city_trees:
    # If city exists, add trees at the end of the existing list
    city_trees[city].extend(new_trees)
    print(f"Updated tree list for {city}: {' '.join(city_trees[city])}")
else:
    # Otherwise, add the new city and the list
    city_trees[city] = new_trees
    print(f"City '{city}' did not exist. Added new record with: {new_trees}")
```

```
elif choice == "7":
    print("Exiting program. Goodbye!")
    break
```

```
else:
    print("Invalid choice! Please enter a number between 1 and 7.")
```

```
if __name__ == "__main__":
    main()
```
